Rewriting Pre-Training Data Boosts LLM Performance in Math and Code

How transform-and-retain corpora beat the best filtered datasets — by 17 points on HumanEval.

Paper: Fujii et al., arXiv:2505.02881v4 (March 2026)

Institution: Institute of Science Tokyo & AIST

Kalhar Mayurbhai Patel | SJSU ID: 019140511 | CMPE 255 — Data Mining

The Data Quality Gap

Why open code & math models still trail closed frontier models

Closed frontier models

Qwen3, DeepSeek-V3, GPT-4 etc.

✓ Excellent code & math performance

✗ Pre-training corpora are NOT released

✗ Open community cannot reproduce or improve

→ Open models stuck behind a wall

Public datasets

The Stack v1/v2, Finemath-4+, Stack-Edu

✓ Open and reproducible

✗ Rule-based extraction from web crawls

✗ Classifier-based filtering DISCARDS noisy data

✗ Inconsistent style, missing context, broken deps

→ Lower ceiling on downstream performance

The Big Idea

> ***Don't throw bad code away. Rewrite it.***

The transform-and-retain paradigm

Filter

Drop only the unfixable (syntax errors, comment-only files)

Rewrite

Use a 70B LLM to fix style, structure, and semantics

Retain

Keep the diversity of real-world code — don't synthesize from scratch

Headline Result

Same Llama-3.1-8B base. Same 50B-token budget. Only the data changes.

+17.0

HumanEval pass@1

vs Stack-Edu

+16.1

HumanEval+ pass@1

vs Stack-Edu

+12.4

GSM8K accuracy

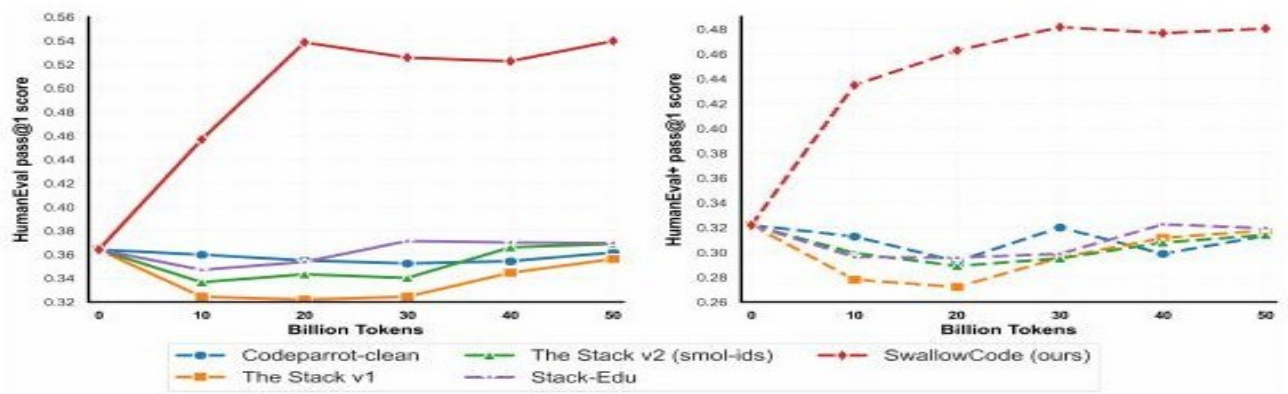
vs Finemath-4+

+7.6

MATH accuracy

vs Finemath-4+

Figure 1 — SwallowCode (red) crushes every comparable open Python corpus



Why Llama-3.1-8B (and not a frontier model)?

> *"Frontier models already saturate code & math benchmarks — additional data yields only marginal gains. That obscures whether improvements come from the dataset or incidental model factors."*

Too strong

Qwen-2.5/3, gpt-oss

Already ~saturated.
Tiny gains hide whether
data even helped.

Just right

Llama-3.1-8B

Strong baseline, but
genuinely improvable.
Gains attributable to data.

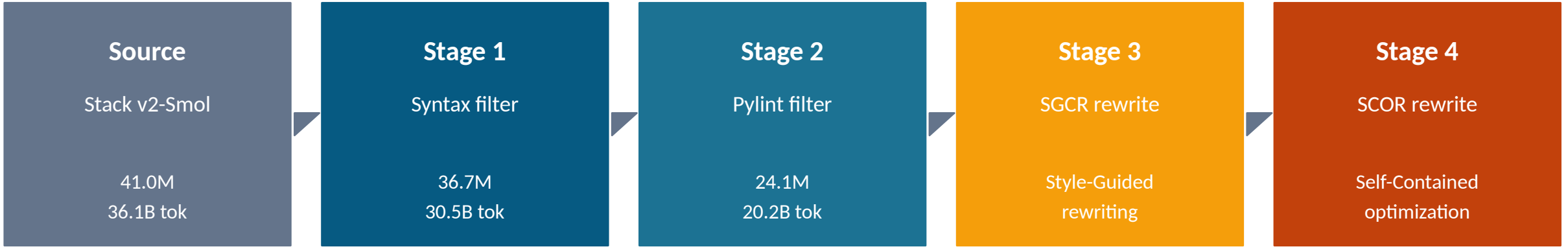
Too weak

Smaller / older models

Low ceiling, noisy.
Results won't transfer to
realistic LLM training.

The Four-Stage SwallowCode Pipeline

From 41M raw Python files → 16.1B tokens of high-quality, self-contained code



Filtering Stages 1 + 2

Stage 1 — Syntax filter:

Compile each snippet with Python's compile(). Drop anything that doesn't parse. (-9.7%)

Stage 2 — Pylint filter (threshold ≥ 7.0):

Style + structure check. Custom comment-ratio penalty kills near-empty 'doc-only' files. (-34.3%)

Rewriting Stages 3 + 4

Stage 3 — SGCR (Style-Guided Code Rewriting):

10-criterion Google Python Style Guide rewrite — names, docstrings, type hints, error handling.

Stage 4 — SCOR (Self-Contained Optimization):

Inline missing deps. Replace inefficient algorithms. Turn trivial snippets into instructive ones.

Filtering: Worth It, But Plateaus Fast

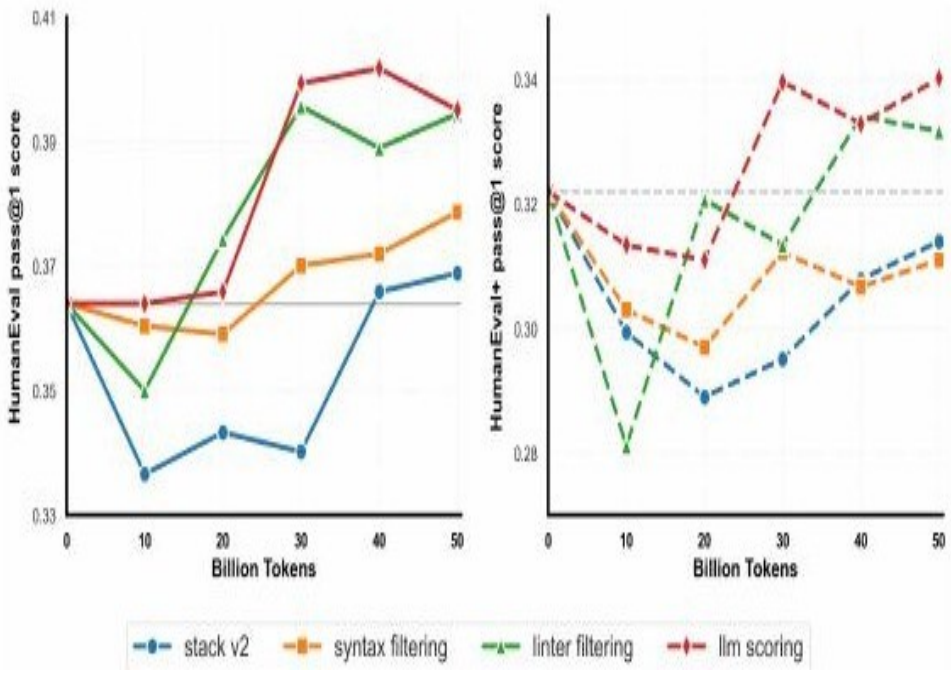


Figure 3 — Filtering on HumanEval / HumanEval+

Key takeaways

✓ **Syntax filter helps consistently**

Removes ~10% of files that don't even compile.

✓ **Linter filter adds ~1 point**

Pylint ≥ 7.0 + comment-ratio penalty.

✗ **LLM-based scoring: barely worth it**

<1 point gain, ~19,500 H100-hours of compute.

→ **Filtering plateaus around +1–2 pts.**

Two-Stage Rewriting: SGCR + SCOR

Why two passes? LLMs drift when juggling 19 directives — split style from semantics.

SGCR — Style-Guided Code Rewriting

Enforces 10 criteria from the Google Python Style Guide:

- Descriptive variable names
- Useful comments + docstrings
- Type annotations
- Modular function design
- Sensible variable scopes
- Proper error handling
- Standard formatting

Effect: +7 to +9 pts on HumanEval

SCOR — Self-Contained Optimization Rewriting

Fixes the three semantic failures of SGCR-only data:

- **Missing dependencies**

Inline imports, satisfy undefined helpers

- **Inefficient algorithms**

Replace $O(n^2)$ with linear / DP solutions

- **Trivial snippets**

Turn print("hi") into instructive examples

Effect: +5 to +6 pts on top of SGCR

Where the Real Gains Come From

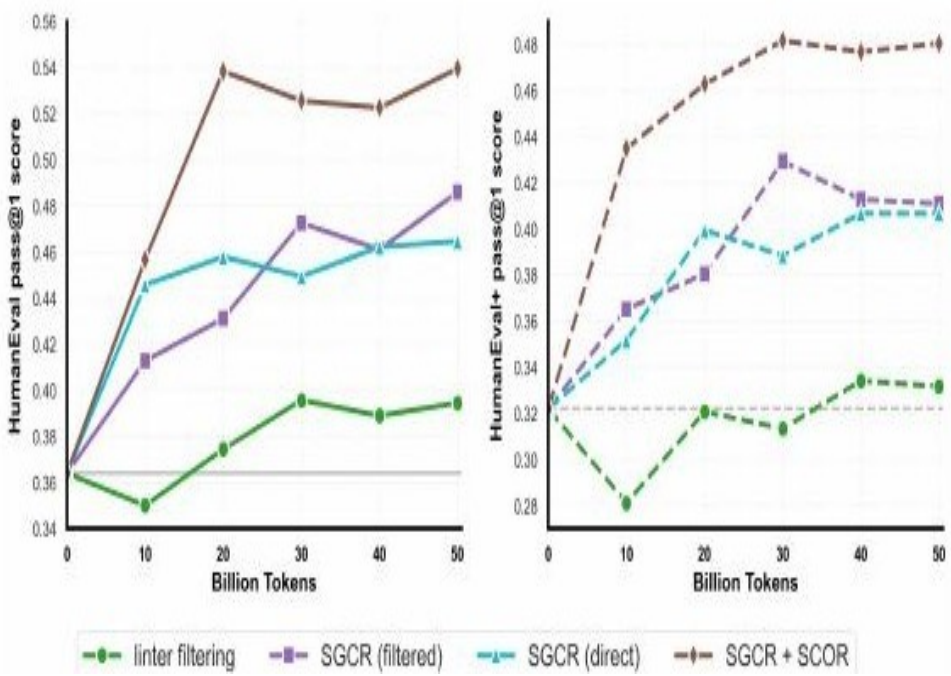


Figure 4 — Rewriting stages dwarf filtering

~14 pts

from rewriting alone

Breakdown (HumanEval pass@1)

Filtering only (Stage 1+2)

+1 to +2 pts

\+ SGCR

+7 to +9 pts

\+ SCOR (on top of SGCR)

+5 to +6 pts

→ Rewriting > 7× more impact than filtering.

A Cautionary Tale: The MBPP Drop

Why SGCR appeared to hurt one benchmark — and what it really tells us

What MBPP expects:

```
def is_Power_Of_Two(x):
    return x and (not(x & (x - 1)))

def differ_At_One_Bit_Pos(a, b):
    return is_Power_Of_Two(a ^ b)
```

What SGCR-trained model writes:

```
def is_power_of_two(x: int) -> bool:
    """Return True if x is a power of 2."""
    return bool(x) and not (x & (x - 1))
```

The verdict: benchmark artifact, not capability regression

MBPP hard-codes function names like `is_Power_Of_Two`. SGCR rewrites them to `snake_case` (PEP-8). Result: harness reports `"is not defined"` — even when the logic is perfect. The authors disclose this honestly and exclude MBPP from evaluation.

Lesson: benchmarks embed assumptions. Always ask what's actually being measured.

SwallowMath: Same Playbook, Math Domain

Apply the transform-and-retain paradigm to Finemath-4+ — does it transfer?

Math rewriting prompt — 5 steps

1. Strip web headers, footers, privacy notices
2. Remove timestamps + irrelevant metadata
3. Restore missing context in incomplete Q/A
4. Rewrite derivations: concise but complete
5. Format clear step-by-step solutions

Steps 1–2 = filtering analog. Steps 3–5 = SGCR/SCOR analog.

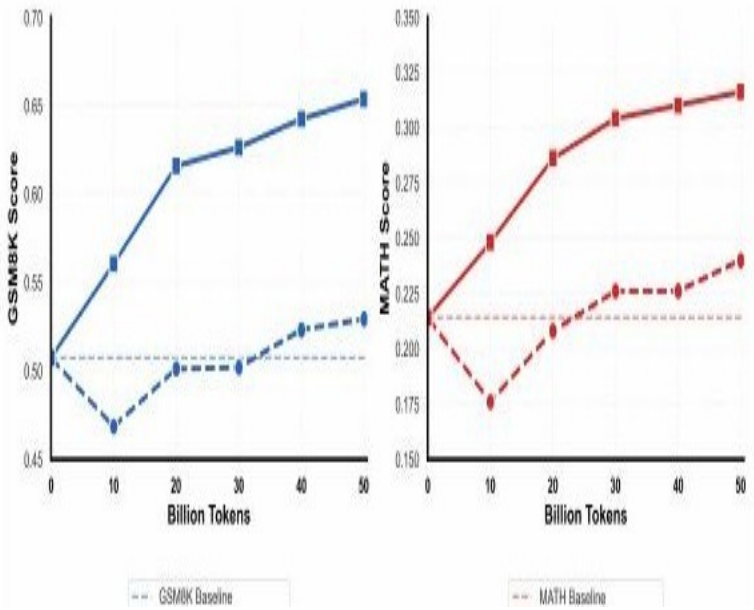
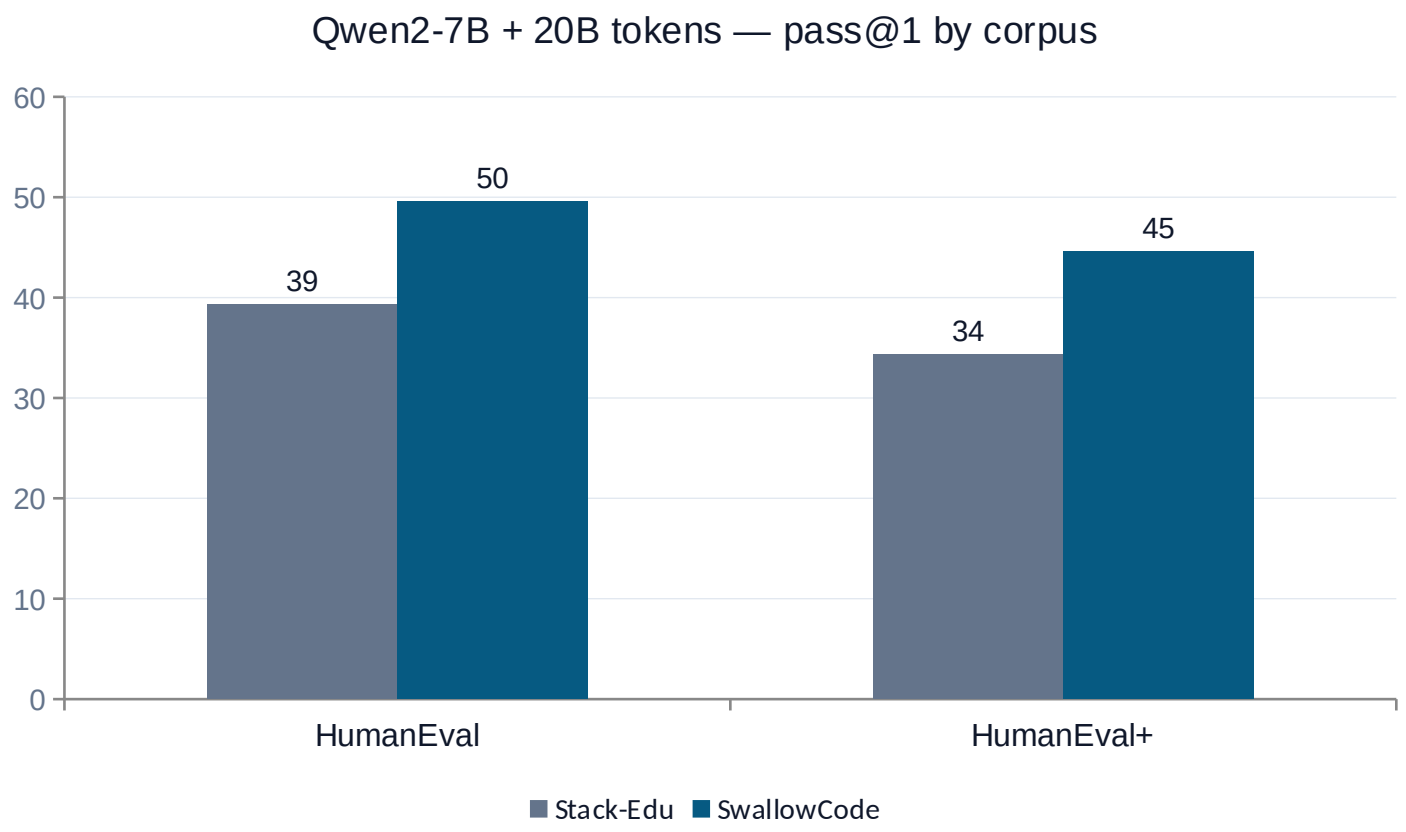


Figure 5 — GSM8K +12.4, MATH +7.6

→ Transform-and-retain transfers beyond code.

Does It Generalize Beyond Llama?

Appendix B — repeat the experiment from a Qwen2-7B base, 20B-token budget



Two big +10.3 lifts

HumanEval: 39.3 → 49.6 (+10.3)

HumanEval+: 34.3 → 44.6 (+10.3)

Same dataset, totally different base model — same dramatic improvement.

→ The pipeline produces genuinely better data, not just data tuned to Llama-3 quirks.

Did They Cheat? (No.)

Decontamination + self-contamination checks (Appendix H)

1. Direct test-set leakage

Streamed entire 16.1B-token corpus. Checked against:

- HumanEval / HumanEval+ prompts (code)
- GSM8K / MATH prompts + solutions (math)

Tests:

- `exact-match search`
- `Jaccard similarity ≥ 0.8`

Result: no matches found.

2. Rewriter self-contamination

Concern: Llama-3.3-70B-Instruct (the rewriter) has Dec 2023 cutoff — could it have seen GSM8K?

Test:

Evaluate on GSM-Plus, released after the cutoff.

Result:

```
Finemath-4+:    35.75
SwallowMath:    46.52
```

Gains hold on data the rewriter has never seen.

Is the Compute Cost Worth It?

Yes — and the asset compounds across the open community

23.7K

H100-hours

to rewrite SwallowCode

1.22×

compute over scoring

but ~10× the gains

16.1B

tokens released

as a community asset

Higher data efficiency

Stack-Edu would need >100B tokens to maybe match SwallowCode at 50B — and trends suggest it never quite gets there. Rewrite once, save tokens forever.

Reusable community asset

Tokyo Tech paid the rewriting cost once. Anyone can download SwallowCode and reuse it for free — across many models, many studies. Cost amortizes across the field.

Limitations the Authors Acknowledge

Source-data biases persist

Rewriting cleans surface noise but inherits distributional biases of The Stack v2 / Finemath-4+. The 70B rewriter also imposes its own preferences (variable naming, solution style).

Only Python tested

Pipeline is in principle language-agnostic — needs only a syntax checker + linter. Multi-language validation deferred to future work due to compute constraints.

50B-token budget

All ablations capped at 50B for controllability. Behavior at 500B / 1T tokens unproven; trends are encouraging but not a guarantee.

Rewriter is the ceiling

Quality of SwallowCode is bounded by Llama-3.3-70B-Instruct. Newer rewriters (Llama 4, gpt-oss, Qwen3) could lift the ceiling — that's both a limitation and an opportunity.

My Take — Five Things That Stick

1

The methodology > the dataset

SwallowCode is great. The transform-and-retain paradigm is the real contribution — generalizable to any language or domain.

2

Data is the new model

As open rewriters improve, every dataset built this way regenerates for free. Compounding returns architecture changes can't match.

3

Filtering plateaus, rewriting compounds

+1–2 pts from filters. ~14 pts from rewriting. The community has been spending its energy in the wrong place.

4

Two-stage prompting beats one-stage

SGCR + SCOR split is a generalizable insight. When you have many objectives, decouple them. Don't stuff into one prompt.

5

Benchmarks are not neutral

MBPP punishing snake_case is a perfect example. Always interrogate what your benchmark is actually measuring.

If You're Training a Code or Math Model in 2026

Practical takeaways from the SwallowCode pipeline

For code

- Don't just filter — rewrite. Even a small rewriting pass on top of filtered data is likely worth it.
- Decouple style from semantics. Two prompts, two passes. SGCR then SCOR.
- Filter first, rewrite second. Both stages compound; rewriting raw garbage wastes 70B tokens of compute.
- Audit your benchmark for naming conventions. MBPP is a cautionary tale.
- Decontaminate aggressively, including against the rewriter — use post-cutoff benchmarks.

For math

- Boilerplate stripping (privacy notices, timestamps) helps more than you'd think.
- Step-by-step solution rewriting is a major source of GSM8K gains.
- Same pipeline as code, just adjust the prompt for math content.

Thank You

Paper

Rewriting Pre-Training Data Boosts LLM Performance in Math and Code

Fujii et al., Institute of Science Tokyo & AIST

arXiv:2505.02881v4 — March 2026

Resources

```
Datasets:  huggingface.co/collections/tokyotech-llm/swallowcode
Pipeline:  github.com/rioyokotalab/swallow-code-math
Medium article:  on my profile (link in repo README)
```

Questions?

Kalhar M. Patel

SJSU ID: 019140511

CMPE 255 — Data Mining

Short Story Assignment

Spring 2026